Universidad Tecmilenio
Maestría en Ciencia de Datos
Gestión de proyectos de inteligencia artificial

Proyecto Fase II

Manual de despliegue en la nube

Contenerización, ejecución local y publicación en PaaS del modelo DEMANDA-IA

Alumno: José Gerardo Ortiz Leñero
Matrícula: _____
Profesora: Ing. Tania Judith Mendoza Torres
Octubre de 2026

Contenido

1. Propósito y alcance 3

2. Arquitectura de la solución 3

2.1 Modelo 4

2.2 Contrato de la API 4

3. Requerimientos técnicos 5

3.1 Para ejecutar en local 5

3.2 Dependencias de la imagen 6

3.3 Recursos medidos 6

3.4 Para la nube 6

4. Estructura del repositorio 6

5. Despliegue local paso a paso 7

5.1 Obtener el código 7

5.2 Revisar el Dockerfile 7

5.3 Construir la imagen 8

5.4 Ejecutar el contenedor 8

5.5 Verificar que funciona 8

5.6 Detener y limpiar 9

6. Estrategia de despliegue en la nube 10

6.1 Modelo de servicio 10

6.2 Modelo de implementación 10

6.3 Proveedor y flujo de entrega 10

7. Despliegue en Google Cloud Run paso a paso 11

7.1 Preparar el proyecto 11

7.2 Subir la imagen 11

7.3 Desplegar el servicio 11

7.4 Verificar en la nube 12

7.5 Programar la corrida nocturna 12

7.6 Actualizar y regresar versiones 12

7.7 Alternativa: Render 12

8. Operación y mantenimiento 13

8.1 Variables de entorno 13

8.2 Monitoreo 13

8.3 Reentrenamiento 13

8.4 Seguridad 13

8.5 Solución de problemas 14

9. Alineación con ISO/IEC 23053 14

10. Uso de herramientas de IA generativa en la documentación 15

Referencias 15

1. Propósito y alcance

Este manual explica cómo construir, ejecutar y publicar DEMANDA-IA, el servicio que pronostica la demanda diaria por SKU y sucursal para Farmacias Occidente Salud. En la Fase I definimos el caso de negocio: una red de 140 sucursales en el occidente de México que hoy reabastece con reglas fijas y que quiere pasar a un pedido calculado cada noche. En esta Fase II el modelo ya existe como servicio y este documento es la guía para que cualquier persona del equipo lo levante sin depender de quien lo programó.

Cubre cuatro cosas: la arquitectura y sus componentes, los requerimientos técnicos, el procedimiento paso a paso (primero en local con Docker y después en la nube) y la operación diaria una vez publicado. Las pruebas y sus resultados están en un documento aparte, el **Documento de validación y pruebas**.

> **Aclaración.** El repositorio usa datos sintéticos generados con un simulador que reproduce los patrones descritos en la Fase I. El procedimiento de despliegue es el mismo con datos reales; lo único que cambia es el origen del historial de ventas, que en producción vendría del ERP.

2. Arquitectura de la solución

Elegí una arquitectura de un solo contenedor con dos caras: una API REST para sistemas (el ERP y la corrida nocturna) y una interfaz web para personas (gerentes de sucursal o el área de compras). Las dos viven en la misma imagen porque comparten el modelo y así se evita tener que configurar CORS y dos despliegues para un proyecto de este tamaño. Si más adelante el frontend crece, se puede separar sin tocar el backend: el archivo `app.js` ya acepta una URL de API distinta.

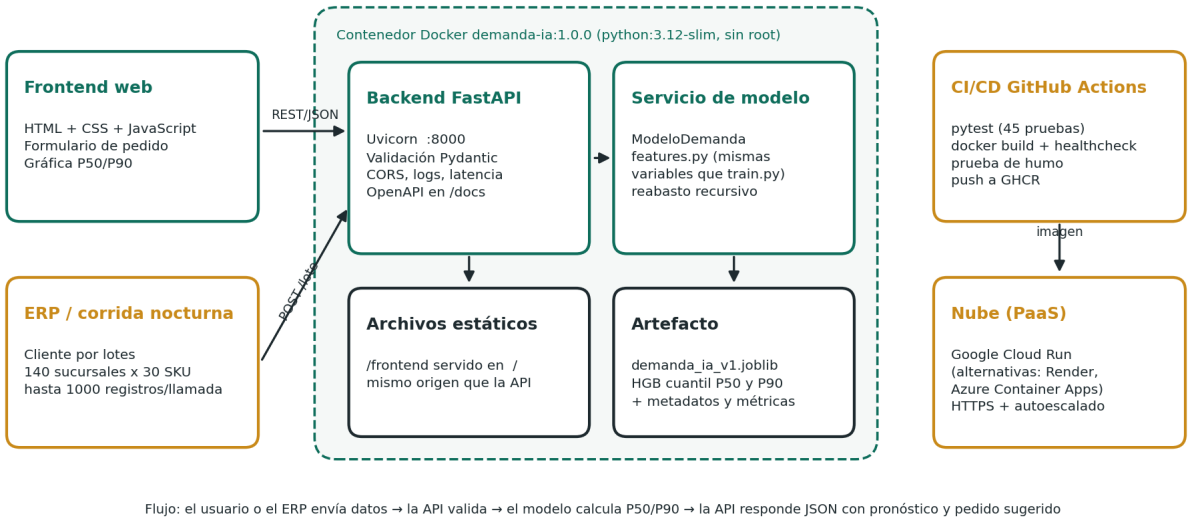


Figura 1. *Arquitectura de DEMANDA-IA: clientes, contenedor, CI/CD y nube.*

Tabla 1. Componentes del sistema

Componente	Ubicación en el repo	Responsabilidad
Simulador de datos	src/demanda_ia/data.py	Genera ventas diarias con estacionalidad, quincenas, festivos, promociones y quiebres de stock
Ingeniería de variables	src/demanda_ia/features.py	Arma las 15 variables de entrada. La usan el entrenamiento y la API, así no hay diferencias entre ambos
Entrenamiento offline	src/demanda_ia/train.py	Entrena los modelos P50 y P90, evalúa y guarda el artefacto .joblib con sus métricas
Servicio de modelo	src/demanda_ia/model.py	Carga el artefacto, revisa entradas atípicas y calcula pronósticos y pedido sugerido
Contratos de la API	src/demanda_ia/schemas.py	Valida tipos, rangos y catálogos con Pydantic; responde 422 si algo no cuadra
Backend REST	src/demanda_ia/api.py	FastAPI con 7 endpoints, log de latencia, CORS configurable y documentación OpenAPI
Frontend	frontend/	Página HTML/CSS/JS sin librerías externas que consume la API
Contenedor	Dockerfile, docker-compose.yml	Imagen multi-etapa, usuario sin root, healthcheck
CI/CD	.github/workflows/ci.yml	Pruebas, build, healthcheck, prueba de humo y publicación en GHCR

2.1 Modelo

El modelo son dos regresores `HistGradientBoostingRegressor` de scikit-learn entrenados con pérdida de cuantiles. El primero estima el cuantil 50 (la venta típica del día) y el segundo el cuantil 90 (un día alto). Con los dos se calcula el pedido: la demanda esperada del periodo más un stock de seguridad que sale de la distancia entre P90 y P50, menos lo que ya hay en anaquel.

Las variables de entrada son de tres tipos: atributos fijos (categoría del producto, tipo de sucursal, estado, tamaño de la tienda, demanda base del SKU), calendario y precio (día de la semana, mes, quincena, festivo, precio relativo a lista, promoción) e historial reciente (venta de hace 7 días, promedio de 7 días y promedio de 28 días). Como el modelo usa atributos de la sucursal y no su identificador, funciona también en sucursales que no estaban en el entrenamiento; esto se comprobó con 10 sucursales apartadas.

2.2 Contrato de la API

Tabla 2. Endpoints expuestos (prefijo /api/v1)

Método y ruta	Entrada	Salida
GET /salud	Ninguna	Estado, modelo cargado, versión, segundos activo

Método y ruta	Entrada	Salida
GET /modelo	Ninguna	Algoritmo, variables, periodo de entrenamiento y métricas
GET /catalogo	Ninguna	140 sucursales, 30 SKUs y límites de uso
GET /historial	sucursal_id, sku, hasta, dias	Ventas diarias simuladas (solo para la demo)
POST /pronostico	sucursal_id, sku, fecha, lag_7, media_7, media_28, precio y promoción opcionales	P50, P90, unidades sugeridas, advertencias
POST /pronostico/lote	Lista de 1 a 1000 solicitudes como la anterior	Lista de resultados y tiempo de cómputo
POST /reabasto	sucursal_id, sku, fecha_inicio, horizonte (1-14), existencia, 28+ días de historial	Pronóstico por día, demanda esperada, stock de seguridad y pedido sugerido

Además, `/docs` muestra la documentación interactiva (Swagger) generada por FastAPI, y `/` sirve la interfaz web. Toda respuesta incluye el encabezado `X-Tiempo-Respuesta-ms`, útil para monitorear desde fuera.

3. Requerimientos técnicos

3.1 Para ejecutar en local

Tabla 3. Software necesario en la máquina de trabajo

Herramienta	Versión probada o mínima	Para qué
Docker Desktop (Windows/macOS) o Docker Engine (Linux)	Docker 24 o superior, con BuildKit	Construir y correr la imagen
Git	2.40 o superior	Clonar el repositorio
Navegador web	Cualquiera reciente	Interfaz y Swagger
curl o PowerShell 5+	El que traiga el sistema	Prueba de humo
Python (opcional)	3.12	Correr pruebas o reentrenar fuera de Docker
Visual Studio Code (opcional)	Cualquiera reciente	Editar y usar la terminal integrada, como en el tutorial del Tema 15

En Windows, Docker Desktop necesita WSL 2 activado. La máquina debería tener al menos 8 GB de RAM y unos 2 GB libres en disco para la imagen base y las capas.

3.2 Dependencias de la imagen

Las versiones están fijas en `requirements.txt`. Esto importa más de lo que parece: un archivo `.joblib` de scikit-learn solo se garantiza compatible con la misma versión con la que se guardó. Si alguien sube la versión de scikit-learn sin reentrenar, el contenedor puede arrancar con advertencias o fallar al cargar el modelo.

Tabla 4. Dependencias de ejecución

Paquete	Versión	Uso
Python (imagen python:3.12-slim)	3.12	Intérprete
fastapi / starlette	0.162.0 / 1.10.0	Framework web
pydantic	2.14.1	Validación de entradas y salidas
uvicorn	0.54.0	Servidor ASGI
scikit-learn	1.8.0	Modelos de cuantiles
numpy / scipy / joblib	2.4.4 / 1.17.1 / 1.5.3	Cómputo y carga del artefacto

pandas, pytest, httpx y matplotlib están en `requirements-dev.txt`. Solo se usan para entrenar, probar y generar reportes, por eso no se instalan en la imagen final.

3.3 Recursos medidos

Con el servicio corriendo con dos workers medí unos 97 MB de memoria por worker y 230 MB en total. El artefacto del modelo pesa 1.8 MB. Un pronóstico individual tarda alrededor de 10 ms y un lote de 1000 registros unos 50 ms. La corrida nocturna completa (140 sucursales por 30 SKU, 4 200 pronósticos) se resolvió en 0.22 segundos. Con esos números, una instancia de 1 vCPU y 512 MB a 1 GB de memoria alcanza de sobra.

3.4 Para la nube

- Cuenta de Google Cloud con facturación activa y la CLI `gcloud` instalada (opción principal).
- Como alternativa, una cuenta de Render o de Azure. Los planes gratuitos y sus límites cambian seguido, así que conviene revisarlos el día que se despliegue.
- Cuenta de GitHub con Actions habilitado. El flujo publica la imagen en GitHub Container Registry (GHCR) con el token que GitHub da a cada ejecución, sin configurar secretos adicionales.

4. Estructura del repositorio

```
demanda-ia/
├── src/demanda_ia/      config, data, features, train, model, schemas, api
├── frontend/            index.html, styles.css, app.js
├── models/              demanda_ia_v1.joblib y demanda_ia_v1.json (metadatos)
├── tests/               test_api_funcional, test_casos_extremos, test_modelo
```

```
├── scripts/             smoke_test.sh, smoke_test.ps1, benchmark.py,
│                        generar_reportes.py, diagrama_arquitectura.py
├── reports/             métricas, sensibilidad, resultados de pruebas, figuras
├── docs/                este manual y el documento de validación
├── .github/workflows/   ci.yml
├── Dockerfile  docker-compose.yml  .dockerignore  .env.example
└── requirements.txt  requirements-dev.txt  README.md
```

El repositorio está publicado en https://github.com/TU_USUARIO/demanda-ia (sustituir TU_USUARIO por el usuario real al subirlo). El README tiene la versión corta de este manual.

5. Despliegue local paso a paso

Los comandos se escriben en la terminal de Visual Studio Code, igual que en el tutorial del Tema 15. Donde PowerShell y bash difieren, se indican las dos variantes.

5.1 Obtener el código

```
git clone https://github.com/TU_USUARIO/demanda-ia.git
cd demanda-ia
```

5.2 Revisar el Dockerfile

El Dockerfile sigue los ocho pasos del tutorial del curso (imagen base, directorio de trabajo, copiar archivos, instalar dependencias, exponer puerto y comando de ejecución) y agrega cuatro mejoras que vale la pena explicar:

- **Tres etapas.** `dependencias` instala los paquetes en un entorno virtual; `pruebas` corre pytest dentro del build; `final` copia solo el entorno virtual y el código. La imagen final no lleva compiladores, pruebas ni pandas.
- **Usuario sin privilegios.** El proceso corre como `appuser` (uid 10001). Si alguien encontrara una falla en la aplicación, no tendría permisos de root dentro del contenedor.
- **HEALTHCHECK.** Cada 30 segundos Docker llama a `/api/v1/salud`. Si falla tres veces seguidas, el contenedor queda marcado como unhealthy.
- **Puerto configurable.** El comando usa la variable `PORT` (8000 por defecto). Plataformas como Render o Cloud Run pueden asignar otro puerto y la imagen se adapta sin cambios.

```
FROM python:3.12-slim AS final
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PATH="/opt/venv/bin:$PATH" \
    PYTHONPATH=/app/src MODEL_PATH=/app/models/demanda_ia_v1.joblib PORT=8000
WORKDIR /app
RUN useradd --create-home --uid 10001 appuser
COPY --from=dependencias /opt/venv /opt/venv
COPY src/ src/
COPY frontend/ frontend/
COPY models/ models/
USER appuser
EXPOSE 8000
```

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 CMD ...
CMD ["sh", "-c", "exec uvicorn demanda_ia.api:app ... --port ${PORT}"]
```

El fragmento anterior es la etapa final; el archivo completo está en la raíz del repositorio. Se revisó con hadolint 2.12 y no dio advertencias.

5.3 Construir la imagen

```
docker build -t demanda-ia:1.0.0 .
```

La primera vez tarda unos minutos porque descarga la imagen base e instala scikit-learn. Al final debe aparecer `naming to docker.io/library/demanda-ia:1.0.0`. Para confirmar que la imagen existe:

```
docker images demanda-ia
```

Si además se quiere que las 45 pruebas automáticas corran dentro del build, se construye la etapa de pruebas. Si una prueba falla, el build se detiene:

```
docker build --target pruebas -t demanda-ia:test .
```

> **Mac con Apple Silicon.** En computadoras con chip Apple (M1 a M4) hay que agregar `--platform linux/amd64` al build si la imagen se va a subir a la nube, porque la mayoría de los servicios corren en procesadores x86.

5.4 Ejecutar el contenedor

```
docker run -d --name demanda -p 8000:8000 demanda-ia:1.0.0
docker ps
```

El parámetro `-d` deja el contenedor en segundo plano y `-p 8000:8000` conecta el puerto 8000 de la máquina con el del contenedor. Unos 20 segundos después, `docker ps` debe mostrar `(healthy)` en la columna STATUS. Los logs se ven con:

```
docker logs -f demanda
```

Ahí debe aparecer la línea `Modelo 1.0.0 cargado` seguida de `Application startup complete`. También se puede levantar con Docker Compose, que lee las variables de `.env.example`:

```
docker compose up --build -d
```

5.5 Verificar que funciona

Primero la prueba de humo, que llama a los siete endpoints y a un caso inválido y termina con error si algo no responde como debe:

```
# Linux, macOS o Git Bash
./scripts/smoke_test.sh http://localhost:8000

# Windows PowerShell
```

```
.\scripts\smoke_test.ps1 -Url http://localhost:8000
```

La salida esperada termina en `Prueba de humo superada`. Después, en el navegador:

Tabla 5. Direcciones de verificación

URL	Qué debe verse
http://localhost:8000/	Interfaz de DEMANDA-IA con la etiqueta verde «API en línea · modelo v1.0.0»
http://localhost:8000/docs	Swagger con los endpoints agrupados en operación, catálogo e inferencia
http://localhost:8000/api/v1/salud	JSON con "estado": "ok" y "modelo_cargado": true

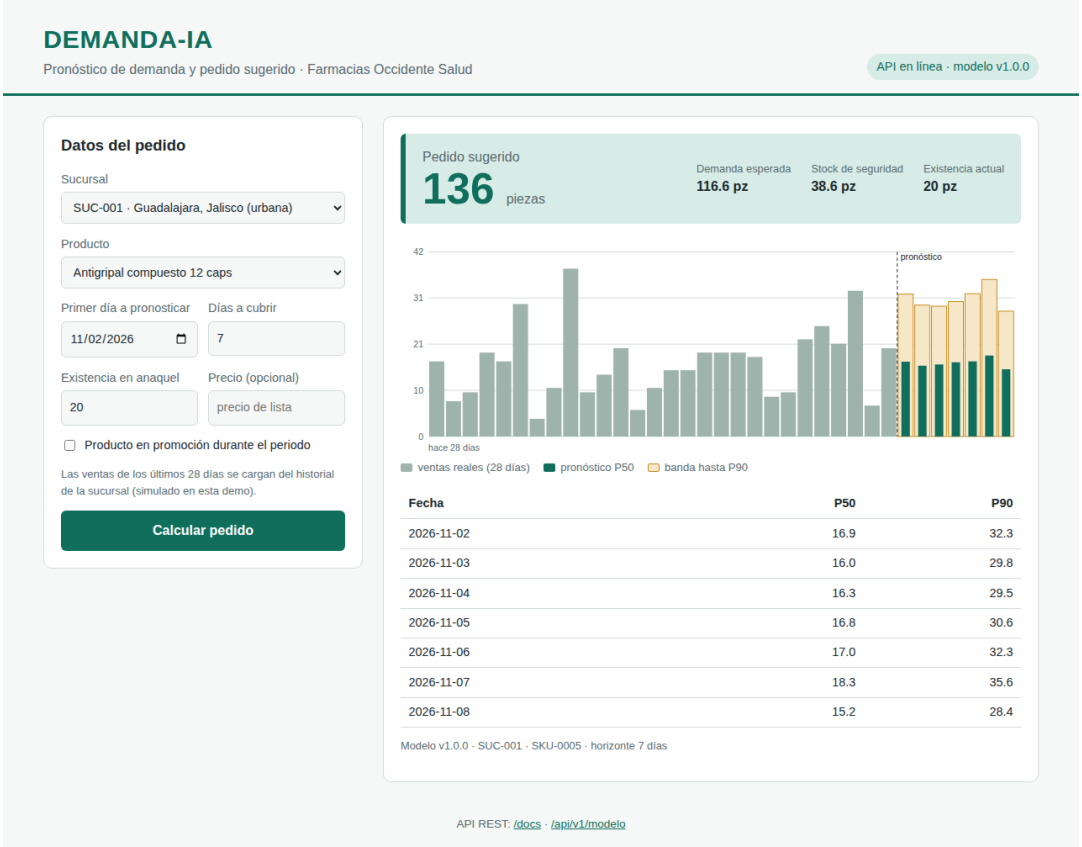


Figura 2. *Interfaz web consumiendo /api/v1/historial y /api/v1/reabasto: antigripal en SUC-001, 7 días, pedido sugerido de 136 piezas.*

5.6 Detener y limpiar

```
docker stop demanda && docker rm demanda
# o, si se usó Compose:
docker compose down
```

6. Estrategia de despliegue en la nube

6.1 Modelo de servicio

En el Tema 16 revisamos IaaS, PaaS, SaaS y FaaS. Comparé las cuatro opciones pensando en lo que DEMANDA-IA necesita: un servicio HTTP pequeño, con carga muy concentrada (la corrida nocturna y consultas esporádicas de día) y un equipo que no quiere administrar servidores.

Tabla 6. Comparación de opciones de despliegue

Opción	Ejemplo	A favor	En contra	Decisión
IaaS	Máquina virtual (Compute Engine, EC2)	Control total	Hay que parchar el sistema operativo, instalar Docker, configurar HTTPS y pagar la VM encendida 24 horas	Descartada
PaaS de contenedores	Google Cloud Run, Azure Container Apps, Render	Usa la misma imagen que ya probamos, HTTPS incluido, escala a cero, cobro por uso	Arranque en frío de unos segundos tras inactividad	Elegida
FaaS	Cloud Functions, AWS Lambda	Pago por invocación	Habría que reescribir la API en funciones y cargar el modelo en cada arranque	Descartada
SaaS de ML	Vertex AI, SageMaker endpoints	Monitoreo de modelos integrado	Costo mayor y más curva de aprendizaje para un solo modelo pequeño	Para una fase futura

La razón de fondo para elegir PaaS de contenedores es que el artefacto que se valida en local es exactamente el que corre en la nube. No hay una segunda configuración que pueda fallar distinto.

6.2 Modelo de implementación

Los datos que usa el servicio son ventas agregadas por día, SKU y sucursal. No incluyen nombres de pacientes, recetas ni otros datos personales, así que una nube pública es aceptable. En producción real el ERP de la cadena probablemente seguiría en su propio centro de datos, lo que da un esquema híbrido: el ERP en casa llama por HTTPS al servicio en la nube durante la noche. Si el área legal pidiera que ningún dato de ventas salga de la red privada, la misma imagen se puede correr en un clúster interno (por ejemplo OpenShift o Kubernetes) sin cambiar el código.

6.3 Proveedor y flujo de entrega

Elegí Google Cloud Run como opción principal porque acepta cualquier imagen de contenedor, da una URL HTTPS al desplegar, escala de cero a varias instancias según las peticiones y cobra solo mientras atiende solicitudes. Render y Azure Container Apps quedan como alternativas con el mismo enfoque.

El flujo completo queda así:

1. El desarrollador hace push a la rama `main` en GitHub.
2. GitHub Actions corre las 45 pruebas con pytest.
3. Si pasan, construye la imagen, la arranca, espera el estado healthy y corre la prueba de humo.
4. Si todo sale bien, publica la imagen en GHCR con las etiquetas `1.0.0` y `latest`.
5. La imagen se copia a Artifact Registry de Google Cloud y se despliega con `gcloud run deploy`. Cada despliegue crea una revisión nueva y se puede regresar a la anterior con un comando.

7. Despliegue en Google Cloud Run paso a paso

Los comandos siguientes usan `PROYECTO` como identificador del proyecto de Google Cloud y la región `us-central1`. Se puede usar otra región; conviene la más cercana a Guadalajara que tenga Cloud Run disponible.

7.1 Preparar el proyecto

```
gcloud auth login
gcloud config set project PROYECTO
gcloud services enable run.googleapis.com artifactregistry.googleapis.com
gcloud artifacts repositories create demanda-ia \
    --repository-format=docker --location=us-central1 \
    --description="Imagenes de DEMANDA-IA"
gcloud auth configure-docker us-central1-docker.pkg.dev
```

7.2 Subir la imagen

```
docker tag demanda-ia:1.0.0
us-central1-docker.pkg.dev/PROYECTO/demanda-ia/api:1.0.0
docker push us-central1-docker.pkg.dev/PROYECTO/demanda-ia/api:1.0.0
```

7.3 Desplegar el servicio

```
gcloud run deploy demanda-ia \
    --image us-central1-docker.pkg.dev/PROYECTO/demanda-ia/api:1.0.0 \
    --region us-central1 --port 8000 \
    --cpu 1 --memory 1Gi --concurrency 40 \
    --min-instances 0 --max-instances 3 \
    --set-env-vars LOG_LEVEL=INFO,MAX_LOTE=1000 \
    --allow-unauthenticated
```

Cloud Run le pasa al contenedor el puerto indicado en `--port` mediante la variable `PORT`, y el CMD de la imagen la usa. Con `--min-instances 0` el servicio no cuesta nada mientras no recibe tráfico; a cambio, la primera petición después de un rato tarda unos segundos en lo que arranca una instancia y carga el modelo. Si eso molesta a los usuarios de día, se sube a 1.

> **Seguridad.** `--allow-unauthenticated` deja la URL pública. Sirve para la demostración académica, pero en producción hay que quitarlo y dar acceso solo a la cuenta de servicio del ERP con el rol roles/run.invoker.

7.4 Verificar en la nube

```
URL=$(gcloud run services describe demanda-ia --region us-central1 --
format='value(status.url)')
./scripts/smoke_test.sh $URL
python scripts/benchmark.py $URL 100
```

La URL tiene la forma `https://demanda-ia-xxxxx-uc.a.run.app`. La misma interfaz web y el Swagger quedan disponibles en esa dirección con HTTPS.

7.5 Programar la corrida nocturna

En producción el ERP llamaría a `/api/v1/pronostico/lote` cada noche. Si se quiere que la nube dispare el proceso, se usa Cloud Scheduler para invocar un job (por ejemplo, un pequeño script en Cloud Run Jobs) que lea las ventas del día, arme los lotes de 1000 registros y escriba el pedido sugerido de vuelta en el ERP. Esa integración depende del ERP real y queda fuera del alcance de esta fase.

7.6 Actualizar y regresar versiones

```
# nueva versión del modelo o del código
docker build -t demanda-ia:1.1.0 . && docker tag ... && docker push ...
gcloud run deploy demanda-ia --image .../api:1.1.0 --region us-central1

# si algo sale mal, regresar el 100 % del tráfico a la revisión anterior
gcloud run revisions list --service demanda-ia --region us-central1
gcloud run services update-traffic demanda-ia --region us-central1 \
    --to-revisions demanda-ia-00001-abc=100
```

7.7 Alternativa: Render

1. En Render, crear un **Web Service** y elegir desplegar desde el repositorio de GitHub (Render detecta el Dockerfile) o desde la imagen `ghcr.io/TU_USUARIO/demanda-ia:latest`.
2. En Health Check Path escribir `/api/v1/salud`.
3. No hace falta configurar el puerto: Render define la variable `PORT` y la imagen la respeta.
4. Al terminar el despliegue, correr `./scripts/smoke_test.sh https://demanda-ia.onrender.com` con la URL que asigne Render.

En planes gratuitos el servicio se duerme tras un tiempo sin tráfico y la primera petición tarda más. Para la presentación del proyecto conviene abrir la URL unos minutos antes.

8. Operación y mantenimiento

8.1 Variables de entorno

Tabla 7. Configuración del contenedor

Variable	Valor por defecto	Descripción
PORT	8000	Puerto en el que escucha Uvicorn
WEB_CONCURRENCY	1	Número de workers. En Cloud Run conviene 1 y escalar con instancias
LOG_LEVEL	INFO	Nivel de detalle de los logs
CORS_ORIGINS	*	Dominios que pueden llamar a la API desde un navegador. En producción, la lista exacta
MAX_LOTE	1000	Máximo de registros por llamada a /pronostico/lote
MODEL_PATH	/app/models/ demanda_ia_v1.joblib	Ruta del artefacto, por si se monta otro modelo

8.2 Monitoreo

- **Disponibilidad.** `/api/v1/salud` se puede usar como verificación de tiempo activo en Cloud Monitoring o en cualquier servicio de monitoreo externo.
- **Latencia.** Cada petición queda en el log con método, ruta, código y milisegundos, y la respuesta lleva el encabezado `X-Tiempo-Respuesta-ms`. En Cloud Run esos logs llegan solos a Cloud Logging.
- **Calidad del pronóstico.** Este es el punto que más importa y el que no se resuelve con infraestructura. Propongo comparar cada semana la venta real contra el P50 pronosticado y alertar si el WAPE semanal pasa de 30 % (la prueba dio 20 %) o si la cobertura del P90 cae debajo de 85 %.
- **Advertencias.** El campo `advertencias` señala historiales en cero, picos atípicos y precios fuera de rango. Si aparecen en muchas respuestas, algo cambió en los datos de entrada.

8.3 Reentrenamiento

Propongo reentrenar una vez al mes o cuando el monitoreo marque degradación. El procedimiento es correr `python -m demanda_ia.train`, revisar que las pruebas de `test_modelo.py` sigan pasando (comparan contra los baselines y revisan la calibración del P90), subir la versión en `config.py` y publicar una imagen con etiqueta nueva. El artefacto anterior queda en la imagen previa, así que regresar a él es cuestión de mover el tráfico.

8.4 Seguridad

- El contenedor corre sin privilegios de root y la imagen final no incluye herramientas de compilación.

- No hay secretos en la imagen ni en el repositorio; `.env` está en `.gitignore`.
- La API valida todo lo que recibe y rechaza campos extra, tipos incorrectos y valores fuera de rango con código 422. Los errores no controlados responden 500 sin mostrar detalles internos.
- Pendiente para producción: autenticación (IAM de Cloud Run o API gateway con llaves), CORS restringido y escaneo de vulnerabilidades de la imagen (Artifact Registry lo ofrece).

8.5 Solución de problemas

Tabla 8. Problemas frecuentes

Síntoma	Causa probable	Qué hacer
`docker build` falla en pip install	Sin internet o proxy corporativo	Revisar la conexión o configurar el proxy en Docker Desktop
Error «port is already allocated»	Otro proceso usa el 8000	Usar `-p 8080:8000` y abrir http://localhost:8080
`docker ps` muestra unhealthy	El modelo no cargó	Ver `docker logs demanda`; revisar que exista models/demanda_ia_v1.joblib
Advertencia de versión de scikit-learn al cargar	Se cambió requirements.txt sin reentrenar	Regresar a la versión fija o reentrenar
El frontend dice «Sin conexión con la API»	Frontend y API en dominios distintos	Definir window.DEMANDA_API y agregar el dominio a CORS_ORIGINS
En la nube la primera petición tarda	Arranque en frío con min-instances 0	Subir --min-instances a 1
Respuesta 422	Entrada inválida	Leer el campo detail: indica el campo y el motivo

9. Alineación con ISO/IEC 23053

La norma ISO/IEC 23053:2022 describe el ciclo de vida de un sistema de IA basado en aprendizaje automático. No obliga a usar un algoritmo o arquitectura en particular, pero pide que cada etapa tenga evidencia. Esta tabla relaciona cada etapa con lo que existe en el repositorio:

Tabla 9. Etapas del ciclo de vida y evidencia

Etapa	Evidencia en DEMANDA-IA
Propósito y requisitos	Caso de negocio de la Fase I; sección 1 de este manual
Adquisición y preparación de datos	Simulador data.py con patrones documentados; en producción, extracción del ERP
Ingeniería de variables	features.py, compartido entre entrenamiento e inferencia, con prueba que verifica que coinciden

Etapa	Evidencia en DEMANDA-IA
Entrenamiento	train.py, separado del servicio, con semilla fija y metadatos guardados junto al modelo
Verificación y validación	45 pruebas automáticas y el Documento de validación y pruebas
Despliegue	Dockerfile, CI/CD y este manual
Operación y monitoreo	Endpoint de salud, logs con latencia, advertencias y propuesta de umbrales (sección 8.2)
Retiro	Versionado de imágenes y revisiones de Cloud Run; para retirar el modelo se elimina el servicio y se conserva la imagen etiquetada como registro

10. Uso de herramientas de IA generativa en la documentación

El Tema 16 propone usar IA generativa para acelerar la documentación técnica. En este proyecto la usé así, y lo dejo escrito porque creo que es parte de documentar con honestidad:

- El código, las pruebas y el primer borrador de este manual y del documento de validación se elaboraron con apoyo de Claude (Anthropic). Las cifras de las tablas salen de la ejecución real de los scripts del repositorio, no de la herramienta.
- La redacción pasó por una revisión para quitar el tono genérico de texto generado y dejarlo en la voz del proyecto.
- Para revisar la estructura contra la norma usé Microsoft Copilot con el siguiente prompt, en la línea del ejemplo del curso:

```
Actúa como especialista en documentación de sistemas de IA. Revisa el manual de
despliegue adjunto del proyecto DEMANDA-IA (pronóstico de demanda por SKU y
sucursal
para una cadena de farmacias, API FastAPI en Docker desplegada en Google Cloud
Run).
Compáralo con las etapas del ciclo de vida de ISO/IEC 23053:2022 e indica qué
secciones faltan o están incompletas en: propósito, datos, verificación y
validación,
despliegue, monitoreo y retiro. Entrega una tabla con hallazgo, sección y
recomendación.
```

> **Pendiente del alumno.** Antes de entregar, sustituir este párrafo por lo que Copilot respondió realmente y por los ajustes que se hicieron a partir de su revisión. Si no se usó Copilot, dejar solo los primeros dos puntos.

Referencias

Chen, M. (2025). ¿Qué es una API (interfaz de programación de aplicaciones)? Oracle. https://www.oracle.com/latam/cloud/cloud-native/api-management/what-is-api/

China, C. y Goodwin, M. (s.f.). IaaS, PaaS, SaaS: ¿cuál es la diferencia? IBM. https://www.ibm.com/mx-es/think/topics/iaas-paas-saas

Docker Inc. (s.f.). Dockerfile reference. https://docs.docker.com/reference/dockerfile/

FastAPI. (s.f.). FastAPI in Containers - Docker. https://fastapi.tiangolo.com/deployment/docker/

Google Cloud. (s.f.). Cloud Run documentation. https://cloud.google.com/run/docs

ISO/IEC. (2022). ISO/IEC 23053:2022 Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML). International Organization for Standardization.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Universidad Tecmilenio. (2026). Tema 15. Contenerizar el modelo con Docker [material de curso].

Universidad Tecmilenio. (2026). Tema 16. Despliegue en la nube [material de curso].